Introduction to graph Algorithm

(After mid sem)
(24 Feb 25)

BFS, DFS → do yourself. (myself) (ourself)

MST: for a given weighted graph, a spanning tree is a tree that ~~teaches~~ touches all the vertices.

→ A MST is a spanning tree with the minimum weight.

(Pseudocode + Time complexity)

Kruskal's Algorithm = $O(V \log V + E \log V) = O(E \log V)$

1. for $v \in V$
   key $v = \infty$
2. key $s = 0$
3. Build minHeap(key)
4. for $i = 1$ to n
   $u \leftarrow$ extract min(key)
5. for every $v \in adj(u)$
6. if key.v > $w(u,v)$
   key.v = $w(u,v)$
   $T(v) = u$.

Prim's Algorithm

1. sort the edges
   $T = \phi$
2. For every edge $e \in E$ in the sorted order
3. If $T \cup \{e\}$ is not cycle
   update $T = T \cup \{e\}$

3 March 2025 , Monday

Data Structure :- DISJOINT SET

* Linked List Representation

Representative of a set :-

Union (x, y)

worst case O(N)

rep(x) ≠ rep(y)
→ link one linked list at the end of the other.
→ update for every z ∈ second linked list
rep(z) = rep(y)

(can be O(N) many unions)

(optimized)

Union (x, y)

rep(x) ≠ rep(y)
if (size(x) > size(y))
link y at the end of x
else link x after y
update for every z ∈ second linked list
size(x) = size(x) + size(y)

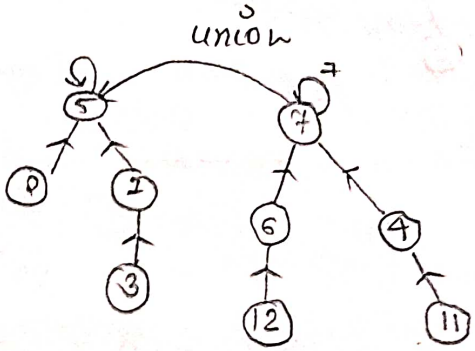
union can be called O(logN) times.
Union can take O(N) time.

Total Time = O(NlogN)

Disjoint Forest

MAKE-SET(x)

```
rep(x)
  if x = parent(x)
      then return(x)
  else
    return rep(parent(x))
```




```
union (x,y)
if ( rep(x) == rep(y))
   return
else
    if (rank(rep(x)) > rank(rep(y))
        Parent (rep(y)) = rep(x).
    else
        parent(rep(x)) = rep(y)
     if (rank(rep(x)) = rank (rep(y)))
         rank(rep(y))+= 1.
```

(Time complexity Analysis):- O(N log N)

Theorem:- If there are m operations of MAKE SET, rep(x), union(x,y), where n are make set operations, then total time taken is $O(m + m \log n)$.

can be improved to

$O(m + m \log^* n)$

IDEA (not part of course)

```
rep(x)
if (x ≠ rep(x)){
rep(x) = rep(rep(x))
}
```

$\log^*(x)$ = how many times to apply log to make it 1

eg:-

$\log^*(2) = 1$

$\log^*(4) = 2$

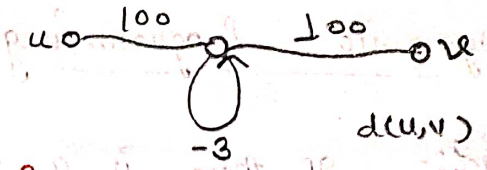
$\log^*(n) = 3, \quad n \le 2^4$

$\log^*(n) = 4, \quad n \le 2^{2^{16}}$

$\log^*(n) = 5, \quad n \le 2^{2^{2^{16}}}$

4/March/2025

Shortest Path



$d(u,v) \Rightarrow$ infinite cycle not possible.

If a graph has negative weight cycle, then shortest path is not defined. leads to $-\infty$

$$d(u,v) = \min_{P(u,v)} \left\{ \sum_{e \in p} c(e) \right\}$$ → does not work on -ve weight cycle.

Single Source Shortest Path

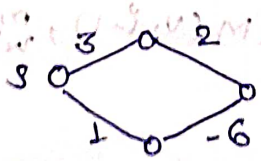
```
for each v ∈ V
    key(v) = ∞
    key(s) = 0
Build minHeap(key)
for i = 1 to n
    u ← extract min(key)
    for every edge (u,w) ∈ E
        if (key(u) + c(u,w) < key(w))
            key(w) = key(u) + c(u,w)
```

6/03/2025

Shortest Path with negative weight

→ If no negative weight, Dijkstra Algorithm works
→ Time complexity $O(E \log V)$

→ If negative weight present, Dijkstra Algorithm fails.

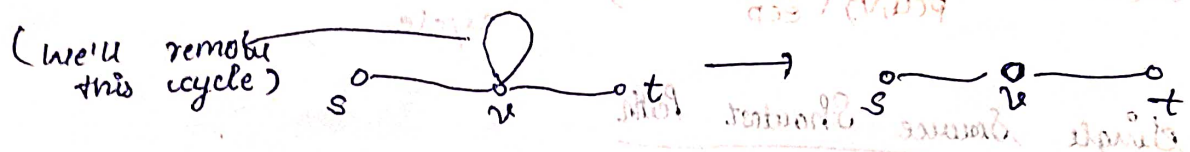


Total weight of cycle is not negative, still dijkstra Algorithm.

Dynamic Programming → define subproblems:

1. Fact:- If there is a negative weight cycle, then shortest path does not exist.

2. Fact:- If there is no negative weight cycle, then the shortest path is simple.



Subproblem :-

$M(v, i)$ = the shortest path from s to v that uses atmost i-edges

$M(s,0) = 0$, $M(v,0) = \infty \ \forall \ v \in V - \{s\}$

$M(v,i) = M(v, i-1)$, if the shortest path $< i$ edges

$= \min_{w|(w,v)\in E} M(w, i-1) + c(w,v)$

↳ cost of $(w$ to $v)$

$M(v, n-1)$ is actual shortest path.

```
for : v ∈ V
    M(v,0) = ∞
    M(s,0) = 0

for i = 1 to n-1 {
    for v ∈ V {
    M(v,i) = min(M(v,i-1), min_{w|(w,v)∈E} (M(w,i-1) + c(w,v)))
    }
}
```

Time complexity :- $O(v \times E)$

worst case :- $O(n^3)$

Space complexity :- $O(n^2)$

Correctness

Lemma:- If there is no negative weight cycle, the Bellman-Ford algorithm returns the shortest path between v to all other nodes.

18/3/2025

Bellman Ford Algorithm

→ finds shortest path from a source S to every other vertices
→ In case of negative weight cycle, it detects it.
→ Time complexity O(VE)
→ Space complexity $O(n^2)$

```
for each v ∈ V
    M(0, v) = ∞   ⟶ Replace with M(v) = ∞
    M(0, s) = 0   ⟶ Replace with M(s) = 0

for (i = 1 to n-1)
    for every vertex v ∈ V
        M(i, v) = M(i-1, v)
        for every edge (w, v) ∈ E
            if M(i, v) > M(i-1, w) + c(w, v)
            then M(i, v) = M(i-1, v) + c(w, v)
```

M(v) > M(w) + c(w,v
M(v) = M(w +
c(w,v

M(v, i) = shortest path from s to v with almost i edges.

* Space complexity can be optimized to O(n)
↓ written with Red colour pen.

optimized - $O(n^3 \log n)$:- Divide and conquer dynamic programming

for each edge $(u,v) \in E$

$M(1,u,v) = c(u,v)$

For each $v \in V$

$M(1,v,v) = 0$

while $(i < n)$ $i = 2i$

for $u, v \in V$

$$M[i,u,v] = \min_{w \in V} [M[i/2,u,w] + M[i/2,w,v]]$$

Logic :—

$M^k(i,j)$ = s.p from i to j with almost k edges

k = almost k edges b/w them

$= M^{k/2}(i,u) + M^{k/2}(u,j)$

$= M^{k-1}(w,j) + c(i,w)$

$M^0(i,j) = c(i,j)$ if $(i,j) \in E$

$= \infty$ o/w

$M^k(i,j)$ = the shortest path i to j where the ~~intersection~~ internal vertices are from the set $\{1,2,\ldots k\}$

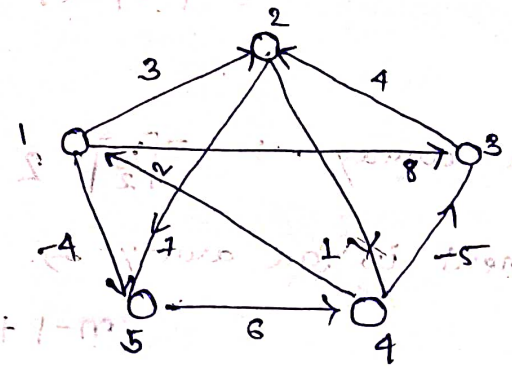
$m^0(i,j) = c(i,j)$ if $(i,j) \in E$

$= \infty$

$= m^{k-1}(i,k) + m^{k-1}(k,j)$

$= m^{k-1}(i,j)$

eg:-



Bellman ford Algorithm

Process :-

$M^1(2,3) = \min(M^0(2,3), M^0(2,1)+M^0(1,3))$

M^0

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

≃

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

$M^{(0)}$

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

≃

$M^{(1)}$

	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	∞
4	2	-1	-5	0	-2
5	∞	∞	∞	6	0

$M^{(2)}$

	1	2	3	4	5
1	0	3	-1	4	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

$M^{(3)}$

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	4	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

$M^{(4)}$

$M^{(5)}$

storing only 2 matrices at a time

enough ⇒ Space comp = $O(n^2)$

20/03/2025

Adversarial Technique

- Find max and min in an array → $3\lceil \frac{n}{2} \rceil - 2$
- Find second largest element in an array → $n-1+\lceil \log n \rceil$

Technique :- A input / strategy is given to our algorithm, so that it works as slow as possible.

#

Unknown

Winner | Losser

useless

⇒ if ($a_i < a_j$)

(1) Both from unknow then ans yes
(2) Both from winner yes
(3) Both from losser yes
(4) Both from useless
(5) If a_i is from winner, keep it winner
(6) If a_i is from looser, keep it looser

* Array of n elements

Let's start algo :-

algo starts → 0th step ... kth step → algo ends

0th step	kth step
|unknown| = n	|unknown| = 0
|winner| = 0	|winner| = 1
|looser| = 0	|looser| = 1
|useless| = 0	|useless| = n-2

Atmost $\lfloor n/2 \rfloor$ comparisons to move all elements to winner and looser set from unknown set.

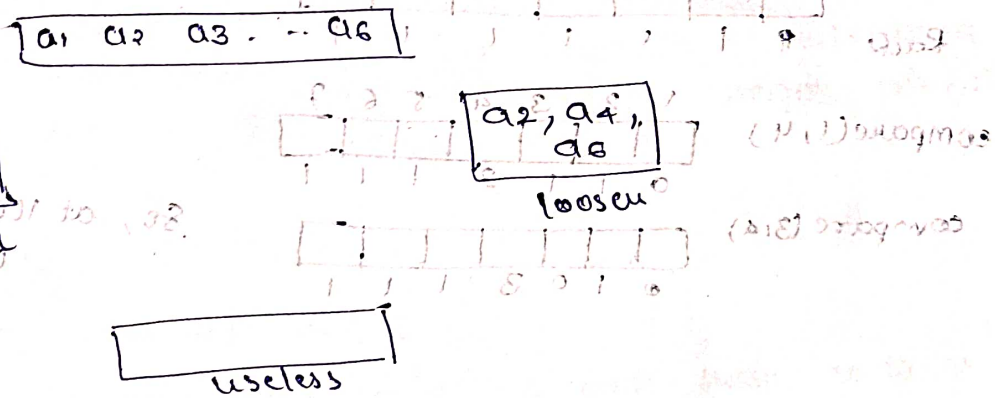
At least $\left\lceil \frac{n}{2} \right\rceil + n - 2 = 3\left\lceil \frac{n}{2} \right\rceil - 2$ comparisons needed to find max and min.

example :-

a_1 a_2 a_3 a_4 a_5 a_6

$a_1 < a_2$
$a_2 < a_3$
$a_3 < a_4$
$a_4 < a_5$
$a_5 < a_6$

$a_1 < a_2$
$a_1 < a_3$
$a_1 < a_4$
$a_1 < a_5$



24th March 2025

Find the 2nd max :- $n + \lceil \log n \rceil - 2$

[Looser Lemma:- for any algorithm A that finds the 2nd max, every non-max element must loose at least once.]

Step1 : for any element $p < m_2$, we say p is dominated by m_2 if
1) either p looses to m_2
2) or p looses to Q and Q is dominated by m_2.

[A comparison where p is dominated by m_2 is called crucial.]

Step2 :- count no. of crucial comparisons and count no of comparisons that involves m_1

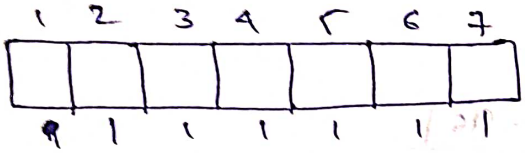
atleast

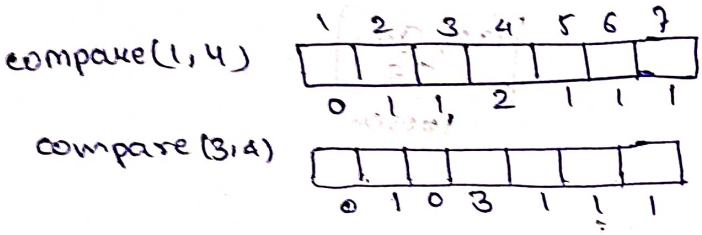
↳ atleast $\lceil \log_2 n \rceil$

claim :- every element $\neq m_2$ must be involved in at least one crucial comparison.

m_1 m_2 m'

m_1+1 m_2 m_2+1





So, at least $\lceil \log_2 n \rceil$ comparison.

Proof of Looser Lemma

→ m_1 m_2

→ m_1 (m_1+1)

Introduction to NP- Completeness

07/10/20

NP → non-deterministic polynomial.

P: The set of problems that can be solved in polynomial time.

NP: The set of problems, for which a solution instance can be verified in polynomial time

(NP is larger set) ⟶ $P \subseteq NP$

Optimization Problem

Decision Problem

↓

Problems which only requires yes or No answer.

(1) D MST :- For given $G = (V, E, W)$, $W: E \to \mathbb{R}$ and a real number Z, whether G has a spanning tree with weight atmost Z.

(2) D Shortest Path: -

Input : $G = (V, E, W)$ u, v, Z

Ouput:- whether there exists a path from u to v weight cost atmost Z ??

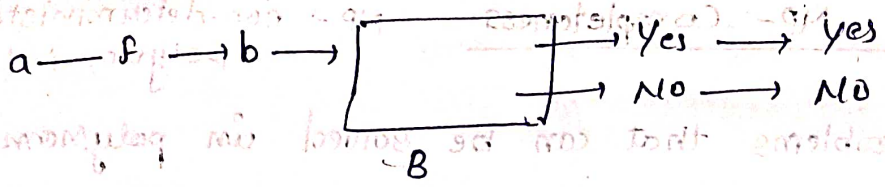
Reduction

¬ (unknown $\xrightarrow{\text{reduction}}$ easy to solve)

↓

(Not easy to solve $\xrightarrow{\text{reduce}}$ unknown prob)

Polynomial Reduction - Let A, B be a decision problems. A polynomial time reduction is a function $f: I(A) \to I(B)$ or equivalently can convert any input of A to an input of B such that a is an yes instance of A iff b is an yes instance of B.



NP-Hard

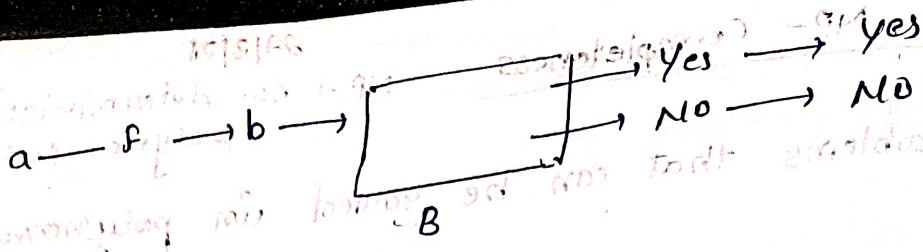
A problem A is said to be NP Hard if for any problem B ∈ NP, there is a polynomial time reduction from B to A.

NP- complete

A problem is NP complete if it is in NP ∩ NP-Hard

A problem is NP complete if some NP-complete problem can have a polynomical time reduction to this problem.

Circuit SAT



NP-Hard

A problem A is said to be NP Hard if for any problem $B \in NP$, there is a polynomial time reduction from B to A.

NP complete

A problem is NP complete if it is in NP ∩ NP-Hard

A problem is NP complete if some NP-complete problem can have a polynomial time reduction to this problem.

Circuit SAT

1st April 2025

The Satisfiability Problem

Input: n boolean variables $x_1, x_2, \ldots x_n$

A boolean function $\phi = c_1 \wedge c_2 \wedge c_3 \ldots \wedge c_m$

$c_i = l_{i_1} \vee l_{i_2} \vee \ldots \vee l_{i_k}$, $k \geq 1$

$l_{ij} \in \{x_1, x_2, \ldots x_n, \overline{x_1}, \ldots \overline{x_n}\}$

Output:- Whether there is a truth assignment of $x_1, \ldots, x_n$ such that ϕ = true.

eg:-

$$\phi = \underbrace{(x_1 \vee x_2)}_{C_1} \wedge \underbrace{(\overline{x_1} \vee \overline{x_2} \vee x_3 \vee x_4)}_{C_2} \wedge \underbrace{(\overline{x_1} \vee x_2 \vee \overline{x_3} \vee x_4 \vee x_5)}_{C_3}$$

Assignment :-

$x_1 = T \quad x_3 = T \quad x_5 = T$

$x_2 = T \quad x_4 = F$

No assignment exist :-

eg:- $\phi = (x_1 \vee x_2) \wedge (x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2})$

$\Rightarrow$ SAT is NP-complete !

Problem $\rightarrow$ 3-SAT Problem. $\rightarrow$ exactly 3 literals in each clause

eg:- $(x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_4}) \ldots\ldots$

Objective :-

Let ϕ be an input of SAT

$= C_1 \wedge C_2 \wedge \ldots\ldots \wedge C_m$

~~Case 1:-~~ For every $i = 1 \ldots n$

Case 1 : C_i has 1 literal.

$C_i = l_j$ (before)

introduce two more literals z_{i1}, z_{i2}

$\tilde{C}_i = (l_j \vee z_{i1} \vee z_{i2}) \wedge (l_j \vee \overline{z_{i1}} \vee z_{i2}) \wedge (l_j \vee z_{i1} \vee \overline{z_{i2}}) \wedge (l_j \vee \overline{z_{i1}} \vee \overline{z_{i2}})$

Case 2 :- C_i has 2 literals.

$C_i = l_{j1} \vee l_{j2}$

introduce one more literal z

$\tilde{C}_i = (l_{j1} \vee l_{j2} \vee z) \wedge (l_{j1} \vee l_{j2} \vee \overline{z})$

Case 3 :- C_i has 3 literals.

no change.

Case 4 :- $|C_i| \geq 4$

$C_i = l_{j_1} \vee l_{j_2} \vee \dots \vee l_{j_k}$

$|C_i| = 4$

$C_i = l_1 \vee l_2 \vee l_3 \vee l_4$

$\tilde{C}_i = (l_1 \vee l_2 \vee z) \wedge (l_3 \vee l_4 \vee \bar{z})$

$|C_i| = 5$

$C_i = l_1 \vee l_2 \vee l_3 \vee l_4 \vee l_5$

$\tilde{C}_i = (l_1 \vee l_2 \vee z_1) \wedge (l_3 \vee \bar{z}_1 \vee z_2) \wedge (l_4 \vee l_5 \vee \bar{z}_2)$

generalized :-

$C_i = l_{j_1} \vee l_{j_2} \vee \dots \vee l_{j_k}$

$\tilde{C}_i = (l_{j_1} \vee l_{j_2} \vee z_1) \wedge (l_{j_3} \vee \bar{z}_1 \vee z_2) \wedge \dots$

$\dots \wedge (l_{j_{k-2}} \vee \bar{z}_{k-4} \vee z_{k-3}) \wedge (l_{j_{k-1}} \vee l_{j_k} \vee \bar{z}$

∴ We can do this in polynomial time.

(Polynomial Time Reduction)

NP complete — NP Hard

3rd April 2025

General Strategy to prove NP-completeness

1. Start with a known NP complete problem π.
2. Show a polynomial time reduction from π to target problem 6.

→ Show yes instance of π ⇒ yes instance of 6

NP instance of π ⇒ No instance of 6.

→ Vertex – Cover

Let $G = (V, E)$ be an undirected graph.
A subset $V' \subseteq V$ is said to be a vertex cover of G if for every edge $(u, v) \in E$,

$$\{u, v\} \cap V' \neq \phi$$

→ Minimum vertex cover problem :-

Find Vertex cover of minimum size

→ D- Vertex cover

For a given graph $G = (V, E)$ and an integer $k > 0$, whether G has a vertex cover of size at most k ?

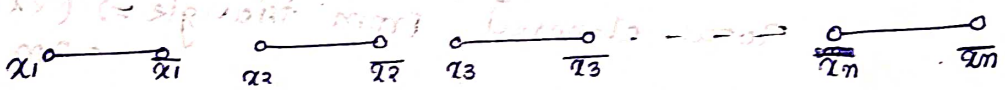
Converting 3 SAT to D Vertex cover

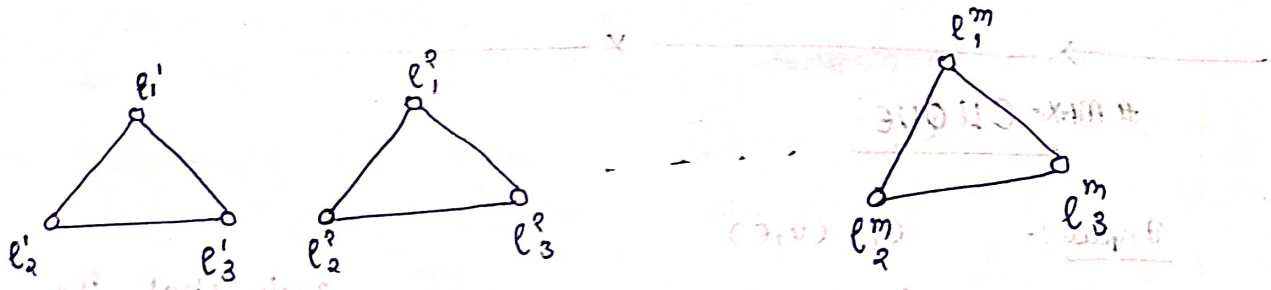
3 SAT $\leq_p$ Vertex Cover

Let $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_m$ be an input to 3SAT

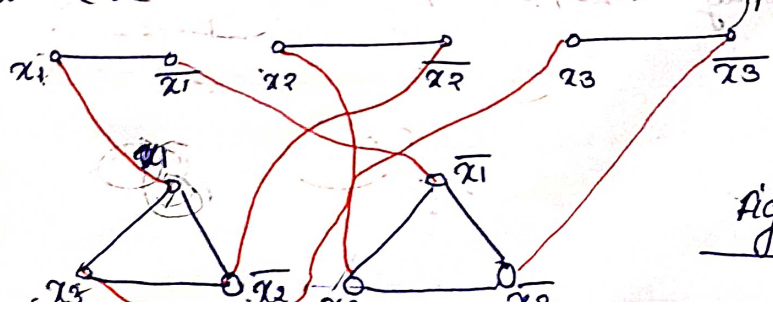
$= (l_1^1 \vee l_2^1 \vee l_3^1) \wedge (l_1^2 \vee l_2^2 \vee l_3^2) \wedge \dots \wedge (l_1^m \vee l_2^m \vee l_3^m)$

where $l_j^i \in \{x_1, \bar{x_1}, x_2, \bar{x_2}, \dots x_n, \bar{x_n}\}$





example :- $(x_3 \vee x_1 \vee \bar{x_2}) \wedge (\bar{x_1} \vee x_2 \vee \bar{x_3}) = \phi = T$



m + m x 2

Fig. 3 SAT

→ For every clause create a triangle, for every variable create a line b/w variable and its complement, then connect the variables of triangle with same variable on line.

$$k = n + 2m.$$

⇝ Let ϕ is satisfiable

⇒ There is a truth assignment of the variable $x_1, \ldots, x_n$

Process:- In fig 3 SAT (Previous Page)

From the lines ⇒ ($\bar{x}_1$ x_1 --- $\bar{x}_n$ x_n)

Pick vertex with positive assignment (Truth variable) and from triangle, i.e each triangle have atleast one truth value vertex (from ϕ), so select the other two vertex to triangle.

Total choosed from triangle ⇒ ($2 \times m$) (m triangle)
$= 2m$

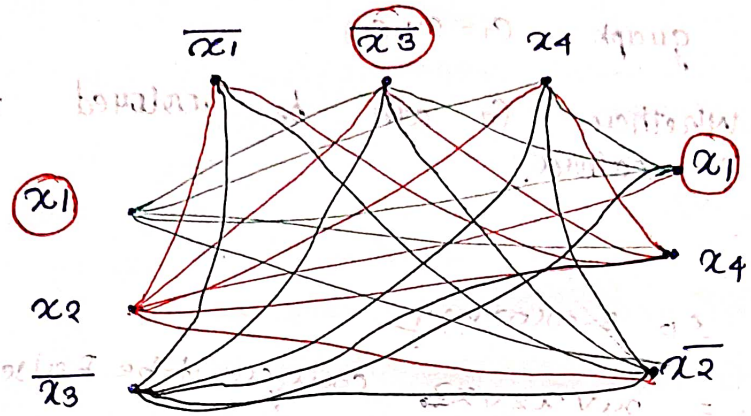
MAX-CLIQUE

Input :- $G_i = (V, E)$

Output :- maximum size subset $V' \subseteq V$ such that for every $a, b \in V'$, $(a, b) \in E$.



→ connect every node of a group to every node of the other group except its complement.



$$\phi = (x_1 \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_1} \vee \overline{x_3} \vee x_4) \wedge (x_1 \vee x_4 \vee \overline{x_2})$$

x_1 $\overline{x_3}$ x_1 forms a clique

7/04/25

Independent Set

Input: An undirected graph $G = (V, E)$

Output: A subset $V' \subseteq V$ s.t. for every $x, y \in V'$, $(x, y) \notin E$

maximum independent set problem

Find the independent set of maximum size

0. Independent Set → for a given graph $G = (V, E)$ and an integer k, whether G has an IS of size at least k?

(G, k) → clique

↓ ↓

(G^c, k) → independent

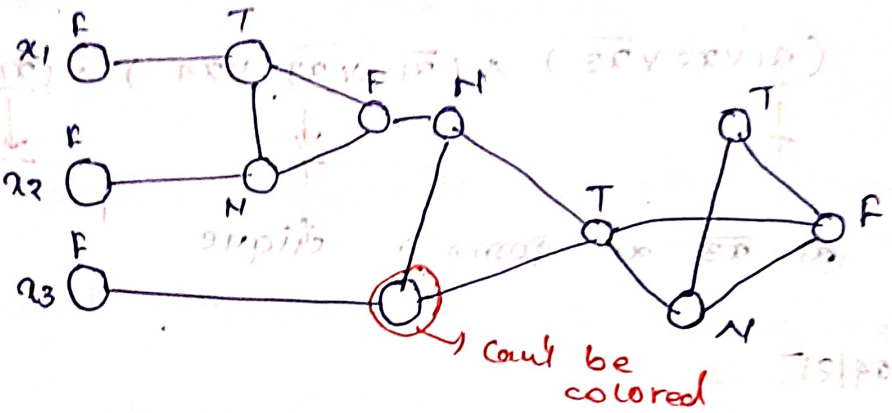
3. coloring

Input:- A graph $G_1 = (V, E)$

Output:- whether G_1 can be colored with 3 colours.

3-SAT $\leq_p$ 3-coloring

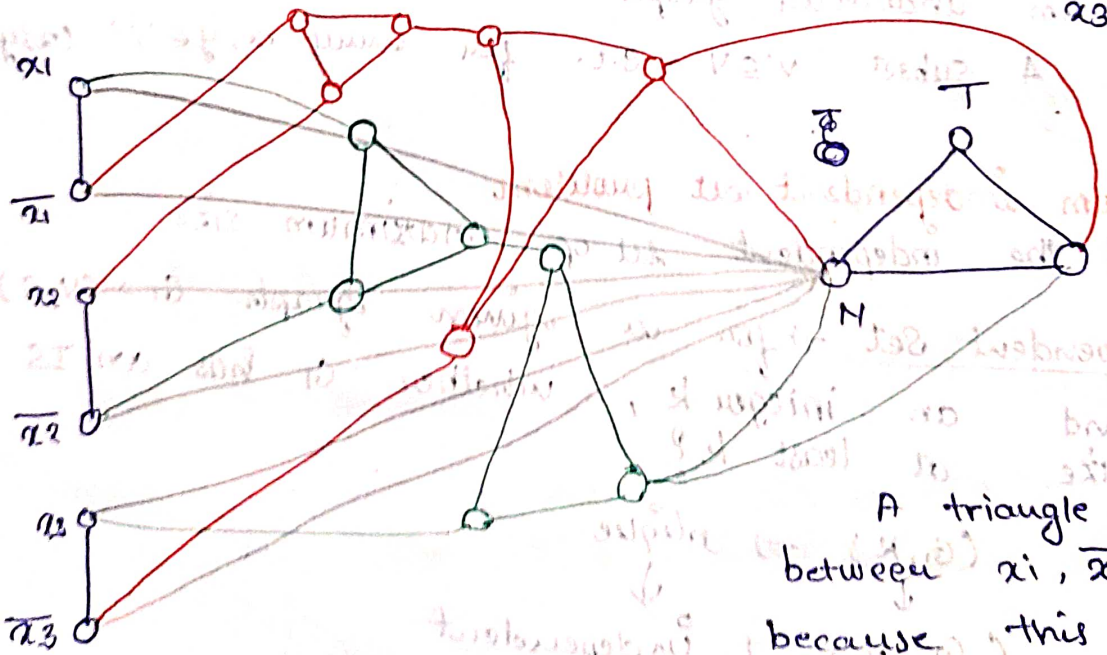
$G_{11} = x_1 \vee x_2 \vee x_3$ (all can't be False F)



Example:-

$(x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee \overline{x_3})$

$x_1 = T$
$x_2 = F$
$x_3 = F$



A triangle is created between x_i, $\overline{x_i}$ and N, because this will force x_i and $\overline{x_i}$ to take values True & false.

Subset Sum

Given a set of integers S, and an integer B, whether there exists a subset $S' \subseteq S$ s.t.

$$\sum_{a \in S'} a = B$$

3-SAT $\leq_p$ SUBSET SUM $(x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee \overline{x}$

	x_1	x_2	x_3	...	x_n	c_1	c_2	c_3
t_1	1	0	0	...	0	1	0	1
f_1	1	0	0	...	0	0	1	0
t_2	0	1	0	...	0	0	0	0
f_2	0	1	0	...	0	1	1	1
t_3	0	0	1	...	0	1	0	0
f_3	0	0	1	...	0	0	1	1
:						:	:	:

08/04/20

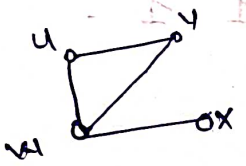
HAMCYCLE

Input:- An undirected graph $G=(V,E)$

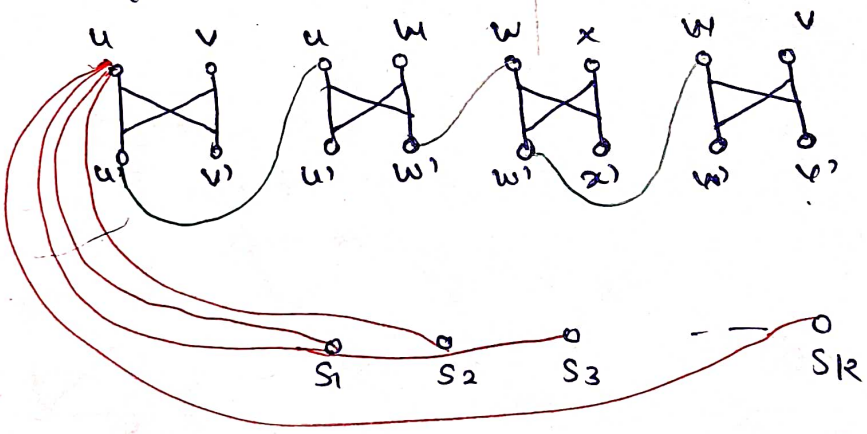
Output:- Yes, if G is hamiltonian
No, otherwise

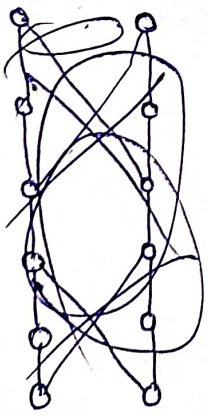
A graph is hamiltonian if it contains a cycle that visit every vertex exactly once.

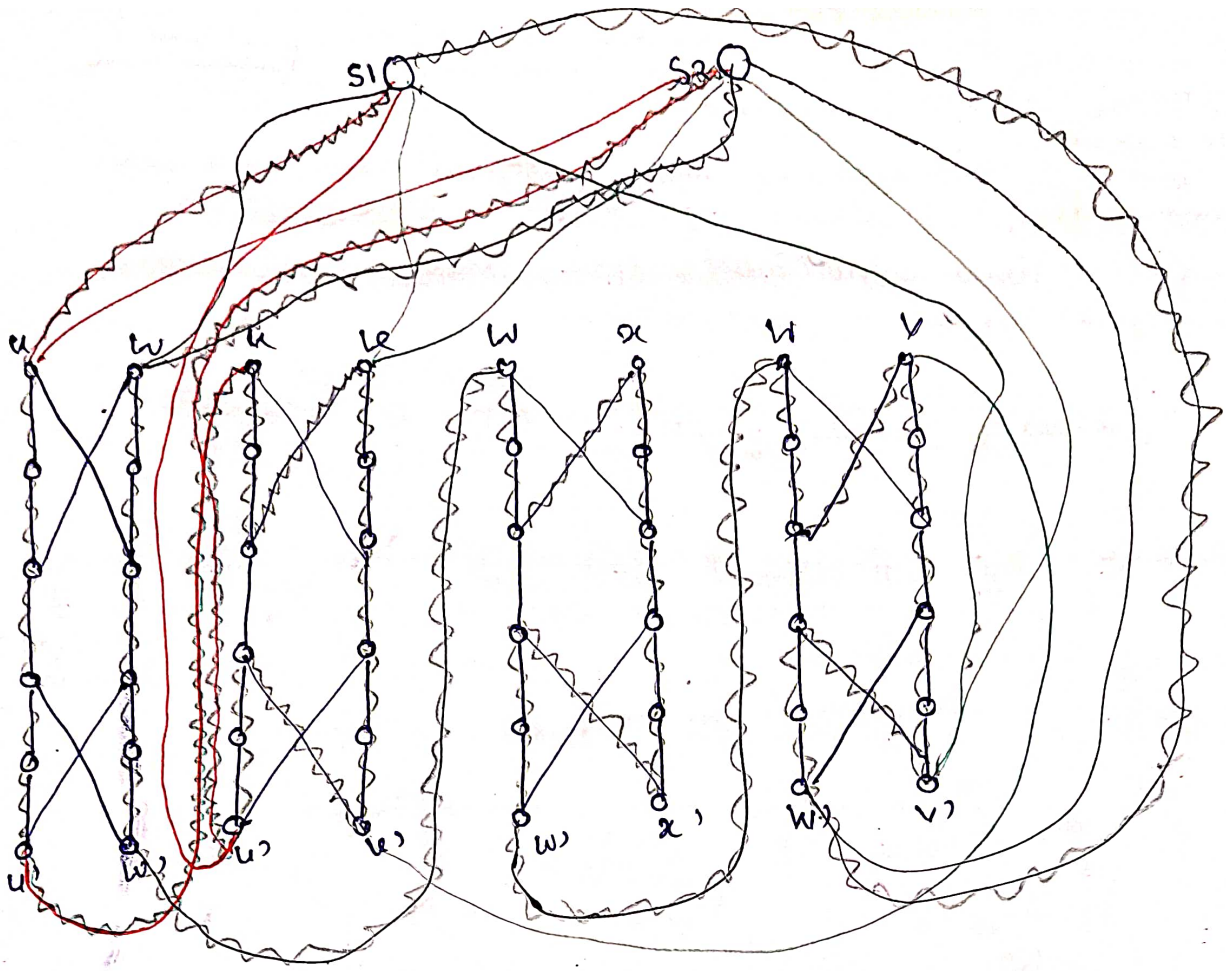
Vertex Cover $\leq_p$ HAMCYCLE $\langle G, k \rangle$



$k=2$







If edge is covered by only one vertex cover ⇒ cover fully

covered by both vertex cover

⇓

cover half

$S_1 \to \begin{pmatrix} u \\ | \\ w \end{pmatrix} \to u' \to \begin{pmatrix} u \\ v \end{pmatrix} \to v \to v' \to u' \to S_2 \to \begin{pmatrix} w \\ | \\ u \end{pmatrix} \to w' \to \begin{pmatrix} w \\ | \\ x \end{pmatrix}$

$\begin{pmatrix} w \\ | \\ x \end{pmatrix} \to \begin{pmatrix} w \\ | \\ v \end{pmatrix} \to S_1$

Travelling Salesman problem.

Input → A complete graph $G_1 = (V, E, W)$ $\quad W \geq 0$ = cost => total time travelled

Output → Smallest cost hamiltonian cycle.

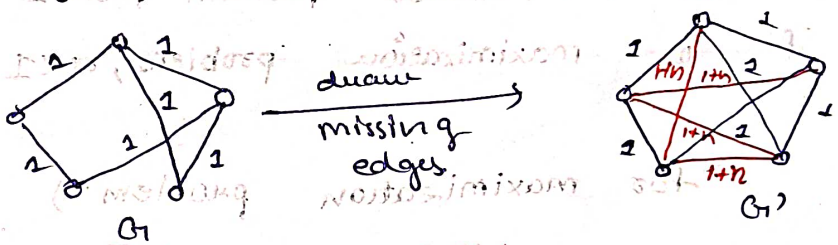
HAMCYCLE $\leq_P$ TSP (Travelling Salesman problem)

D.TSP

Que: whether G_1 has an hamiltonian cycle of cost $\leq k$?

(G_1, k given)

Let G_1 be an input to hamcycle



In G_1' for every edge $(u,v) \mid (u,v) \in G_1$

$w(u,v) = 1$

otherwise $w(u,v) = 1+n$

(to make edges with any cost more than n)

$k = n$

15 April 2025

Approximation Algorithms

Let P be an optimization (minimization) problem, and I be an input of P.

Let A be an algorithm that finds a feasible solution of every input I of P.

Then A is said to be an α approximation algorithm of P if $\frac{A(I)}{opt(I)} \leq \alpha \quad \forall I$.

~~If P is minimization problem, $\alpha > 1$~~

~~If P is maximization problem, $\alpha < 1$~~

for maximization problem $\Rightarrow$

$$\frac{A(I)}{opt(I)} \geq \alpha \quad \forall I$$

TSP → Travelling Salesman Problem

Input :- A complete graph $G = (V, E, W)$

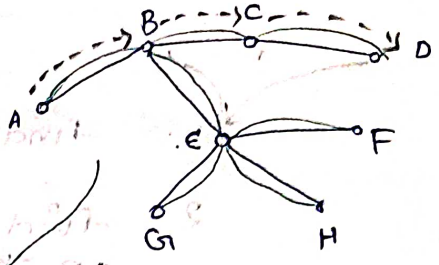
Output :- Hamiltonial cycle with smallest cost

Metric - TSP

Input :- A complete graph G with metric edge weight.

Metric Edge Weight :- The edge weight satisfies triangle inequality

Observation :- Let OPT be the cost of the optimal TSP tour. $W(MST) \leq OPT$



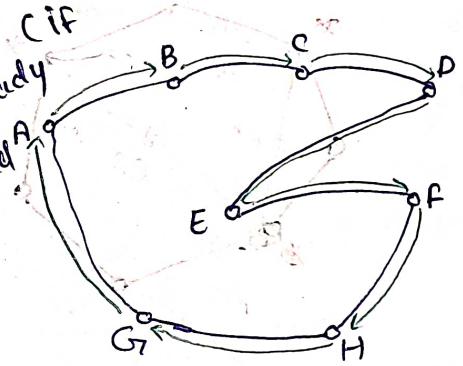
A B BC CD DC CB BE EF
FE EH HE EG GE EB BA

Blue ⇒ original
green ⇒ add one more edge so that degree of each node become even and graph become eulerian

Nodes visited → Sequence: A B C D E F H G A $\leq 2 \cdot MST$

$\leq 2 \cdot OPT$

start from A, follow the path and make the nodes visited (if they are not already visited), if visited skip it.



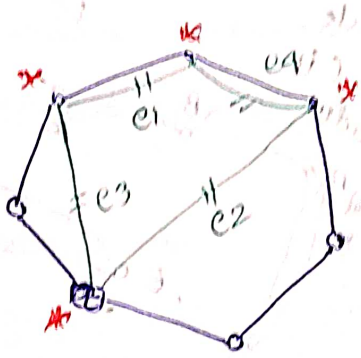
Algorithm :-

1. Find an MST T of G.
2. Find an eulerian tour on the graph T, which is constricted from T by doubling every edge.
3. Find a TSP tour from the Eulerian tour by short cutting.

⇒ If edge weights are not metric, then a cannot be defined

Modified Algorithm (If there exist some nodes which already have even degrees)

1. Find an MST of G.
2. Find a minimum cost matching M on the odd degree vertices of T.
3. Find eulerian tour on T U M
4. Find a TSP tour from the eulerian tour by shortcutting



→ odd degree vertices

{e1, e2}
{e3, e4}

Vertex-Cover

Input → A graph G = (V, E)

Output → Vertex cover of minimum size.

Algorithm

1. S = ϕ
2. E' = E
3. until E' ≠ ϕ
   Pick e = (a, b) ∈ E'
   S = S ∪ {a, b}
   Delete all the edges in E' that has either a or b as its end point

Fact :- Let M be any Matching in G
Then OPT $\geq |M|$.

Our solution $2m \leq 2 \cdot OPT$

17th April

0-1 knapsack

Input :- n items with value $v_1, v_2, \ldots v_n$
weight $w_1, w_2, \ldots w_n$

and a knapsack with capacity W.

Output :- Subsets of items $i_1, i_2, \ldots i_k$ s.t

$\sum_{j=1}^{k} w_{ij} \leq W$ and $\sum_{j=1}^{k} v_{ij}$ is maximized

Let opt' be the optimal solution of fractional knapsack, then opt $\leq$ opt'

---

1. Sort with respect to $\frac{v}{w}$ value
   1.1 $i = 1$
2. while the bag has capacity
   pick item i
   $w' = w - w_i$
   $M = M \cup \{i\}$
   i++
3. If $v_i > \sum_{j=1}^{i-1} v_j$, then $M = \{i\}$
4. Return M.

$$opt' = \sum_{j=1}^{i-1} v_j + \frac{(W - \sum w_i) \times v_i}{w_i}$$

$$opt' \leq \sum_{j=1}^{i-1} v_j + v_i$$

$$\max\left\{\sum_{j=1}^{i-1} v_j, v_i\right\} \geq \frac{opt'}{2} \geq \frac{opt}{2}$$

$$Sol \geq \frac{opt}{2}$$

Bin - Packing

Input:- n items with weight $a_1, a_2, \ldots a_n \leq 1$

Bins with capacity 1.

Output:- Pack the items with minimum number of Bin.

example:- $\{a_1 \ldots a_n\} = \{0.1, 0.8, 0.5, 0.2, 0.6, 0.2\}$

for $i = 1$ to n

if the ith item can be packed inside one of the existing bin, then pack it, else open a new bin.

Let opt be the optimal number of bins

then $opt \geq \sum a_i$ —— (1)

Observation:-

No two bins can have total content (weight) less than half capacity.

At least $k-1$ bins have items $\geq 1/2$ amount.

$$\sum a_i = b_1 + b_2 + \cdots + b_k$$

$$\sum a_i > \frac{k-1}{2}$$

$$k < 2\sum a_i + 1$$

$$k < 2\,opt + 1$$

21st April 2025

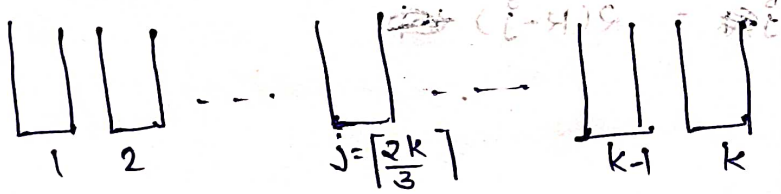
3/2 Algorithm for bin packing

Algorithm

1. Sort the items in the decreasing order of their size.
2. $j = 0$
3. for $i = 1$ to n
   if the item i can be packed in one of the existing bin then pack
   else open a new bin and place item i into the new bin
   $j = j + 1$

Let k be the output of the algorithm

Consider the $\lceil \frac{2}{3}k \rceil^{th}$ bin



Case1 :- Bin j contains an item with size $> 1/2$

$$opt \geq j \Rightarrow opt \geq \frac{2k}{3} \Rightarrow k \leq \frac{3\,opt}{2}$$

case 2 :- all the items in bin j of size $\leq 1/2$ (from jth bin)

$\geq 2(k-j)+1$



b_1

b_2

b_3

$j-1 < 2(k-j)+1$

$opt \geq j-1$

$opt \geq 2(k-j)+1$

↓ combining them

↓

$opt \geq \min\{j-1, 2(k-j)+1\}$

$opt \geq j-1$

$opt \geq j$

$opt \geq \frac{2k}{3}$

$k \leq \frac{3}{2} opt$

$opt \geq 2(k-j)+1$

$opt \geq 2(k-\frac{2}{3}k)+1$

$opt \geq \frac{2k}{3}+1$

$opt \geq 2k/3$

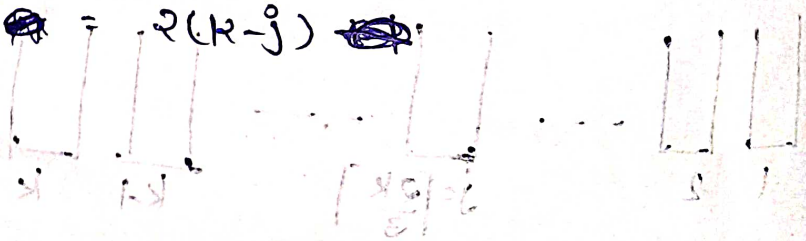
$k \leq \frac{3}{2} opt$

why we choosed $j = 2k/3$

↓

Beause only for $j = 2k/3$,

$j = 2(k-j)$



Theorem:- It is not possible to design an approximation algorithm with factor α, for any $\alpha < 3/2$ for Bin packing problem, unless P = NP

equivalent statement

Designing an α- approximation algorithm for Bin packing for $\alpha < 3/2$ is NP- hard.

Set- Partition Problem

Given integer values $a_1, a_2, \ldots, a_n$ is it possible to partition $S = \{a_1, \ldots a_n\}$ into two subsets S' to S'' such that $\sum_{a \in S'} = \sum_{a \in S''} a$

Let S be an input of set partition

$$\sum_{a \in S} a = 2k$$

let $b_i = \frac{a_i}{\frac{\sum a_i}{2}} = \frac{a_i}{k}$

{why?} if > 1 then total sum will be more than $2k$.

claim:- for all i, $b_i \leq 1$

Suppose that there exists an α- approximation algorithm A, for Bin Packing Problem ($\alpha < 3/2$)

(Set partition of S exist iff the items $b_1 \ldots b_n$ can be covered using 2 bins.)

Apply A on input $b_1, \ldots, b_n$

if set- partition is yes then output of $A \leq 2 \cdot \alpha$

$< 2 \cdot 3/2$

$< 3 \leq 2$

If set partition is No, then output of $K > 2$